ОГЛАВЛЕНИЕ

ГЛАВА 1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ....4
1.1 Наименование....4
1.2 Основание для разработки....4
1.3 Исполнитель....4
1.4 Цель разработки....4
1.5 Содержание работы....5
1.5.1 Задачи, подлежащие решению....5
1.5.2 Требования к архитектуре АСОИ....5
1.5.3 Требования к входным данным....5
1.6 Этапы разработки....6
1.7 Техническая документация, предъявляемая по окончании работы....6
1.8 Дополнительные условия....7
ГЛАВА 2. НАУЧНО-ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ....8
2.1 Постановка задачи проектирования....8
2.2 Описание предметной области....10
2.3 Анализ аналогов....11
2.4. Перечень задач, подлежащих решению в процессе разработки....14
2.5. Обоснование выбора инструментов и платформы для разработки....16
ГЛАВА 3. ПРОЕКТНО-КОНСТРУКТОРСКАЯ ЧАСТЬ....17
3.1 Разработка структуры приложения....17
3.2. Разработка алгоритмов обработки информации....18
3.3. Реализация функционирующего приложения....19
3.3.1 Реализация модуля UDP-соединения....19
3.3.2 Реализация модуля управления виртуальным сетевым интерфейсом....21
3.3.3 Реализация модуля управления событиями ввода-вывода....23
3.3.4 Реализация модуля управления сессиями....24
3.3.5 Реализация модуля аутентификации....25

3.3.6 Реализация модуля шифрования.....27
3.3.7 Реализация клиентской части.....28
3.3.8 Реализация серверной части.....29
3.3.9 Реализация модуля парсинга конфигурационных файлов.....31
3.3.10 Реализация модуля командной строки.....32
ГЛАВА 4. ПРОЕКТНО-ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ.....35
4.1 Тестирование и отладка макета рабочей программы.....35
4.2 Разработка руководства пользователя.....37
4.2.1 Общие сведения.....37
4.2.2 Требования к технической составляющей и архитектуре.....38
4.2.3 Работа с CLI.....39
4.2.4 Фомирование конфигурационного файла.....40
4.2.5 Тестирование.....42
4.2.6 Безопасность.....45
ЗАКЛЮЧЕНИЕ.....46
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....47

ГЛАВА 1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

По дисциплине «Сетевые технологии в автоматизированных системах обработки информации и управления».

1.1 Наименование

«Клиент-серверное приложение реализующее работу по обеспечению инструментов создания и администрирования частной виртуальной сети»

1.2 Основание для разработки

Разработка ведется в рамках курсовой работы по дисциплине «Сетевые технологии в автоматизированных системах обработки информации и управления».

В рамках курсовой работы планируется разработать клиент-серверное приложение реализующее работу по обеспечению инструментов создания и администрирования частной виртуальной сети.

Заказчик: КФ МГТУ им. Н. Э. Баумана.

1.3 Исполнитель

Разработчик: Ли Р. В. Студент КФ МГТУ им. Н. Э. Баумана группы ИУК5-62Б.

Ответственный за приемку работы: Комиссия кафедры ИУК5.

Плановый срок начала работы: 11 марта 2025 года.

Плановый срок окончания работы: 12 мая 2025 года.

1.4 Цель разработки

Разработка клиент-серверного приложения, реализующего следующие задачи:

- Создание частной виртуальной сети.
- Управление параметрами частной виртуальной сети.

- Обеспечение защищенного сетевого соединения внутри виртуальной сети.
- Обеспечение двусторонней аутентификации клиентов и сервера.

1.5 Содержание работы

1.5.1 Задачи, подлежащие решению

1. Разработка механизма перез сеть с использованием протокола UDP.
2. Создание и настройка виртуального сетевого интерфейса TUN.
3. Разработка модуля шифрования передаваемых данных.
4. Реализация механизма аутентификации клиентов и сервера.
5. Разработка клиентской и серверной частей приложения.
6. Обеспечение взаимодействия всех компонентов в едином программном комплексе.

1.5.2 Требования к архитектуре АСОИ

- Модуль работы с сокетами
- Модуль виртуального туннелирования
- Модуль шифрования
- Модуль аутентификации
- Модуль управления сессиями
- Клиентская часть
- Серверная часть
- Модуль интерфейса командной строки

1.5.3 Требования к входным данным

Серверная часть:

1. Виртуальный IP адрес
2. Маска виртуальной сети
3. Порт прослушивания

4. Приватный ключ
5. Список клиентов
    1. Виртульный IP адрес клиента
    2. Публичный ключ клиента

Клиентская часть:

1. Виртуальный IP адрес
2. Приватный ключ
3. IP адрес сервера
4. Порт прослушивания сервера
5. Публичный ключ сервера

1.6 Этапы разработки

1. Анализ требований и проектирование системы
2. Разработка модуля работы с сокетами
3. Разработка модуля виртуального туннелирования
4. Разработка модуля шифрования
5. Разработка модуля аутентификации
6. Разработка модуля управления сессиями
7. Разработка клиентской часть
8. Разработка серверной части
9. Разработка модуля интерфейса командной строки
10. Тестирование и отладка
11. Документирование API
12. Защита курсовой работы

1.7 Техническая документация, предъявляемая по окончании работы

- Техническое задание (в соответствии с ГОСТ 34.602-89).
- Расчётно-пояснительная записка.

- Графические материалы.
- Руководство пользователя.
- Руководство модератора.
- Исходный код с комментариями.
- Отчет о тестировании.
- Презентация проекта.

1.8 Дополнительные условия

Возможность дальнейшего расширения (подключение других сервисов и сайтов объявлений для последующей доставки товаров в пункт выдачи заказов).

Система должна корректно работать и отправлять сообщения в случае ошибки или некорректного внесения данных.

ГЛАВА 2. НАУЧНО-ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ.

2.1 Постановка задачи проектирования

Актуальность и обоснование проекта

В условиях стремительного роста числа удалённых сотрудников, увеличения объёмов передаваемых данных и распространения облачных технологий возрастает потребность в надёжных и гибких средствах организации защищённого взаимодействия между устройствами в распределённых сетях. Частные виртуальные сети (VPN) становятся ключевым инструментом для обеспечения конфиденциальности, целостности и доступности передаваемой информации в недоверенной среде, такой как Интернет.

Однако использование готовых решений зачастую сопряжено с рядом ограничений: отсутствием гибкости настройки, избыточным функционалом, закрытостью исходного кода или высокой стоимостью. В связи с этим особенно актуальной становится задача разработки собственного клиент-серверного приложения, обеспечивающего создание и администрирование частной виртуальной сети с возможностью тонкой настройки, расширяемости и встроенной криптографической защиты.

Проект направлен на реализацию системы, включающей в себя все основные компоненты современной VPN-архитектуры: модуль установления соединений (на основе протокола UDP), механизм туннелирования трафика (через TUN-интерфейс), поддержку шифрования данных, аутентификацию клиентов, управление сессиями и интерфейс командной строки (CLI) для администрирования. Такое решение позволит обеспечить защищённый канал связи между узлами сети, при этом сохранив контроль над архитектурой и возможностью доработки системы под конкретные требования.

Таким образом, разработка данного программного обеспечения обладает как практической значимостью, так и учебной ценностью, обеспечивая углублённое понимание принципов сетевого взаимодействия, криптографической безопасности и архитектурных подходов к построению распределённых систем.

Формулировка задачи

Целью курсовой работы является разработка клиент-серверного программного обеспечения, обеспечивающего создание и администрирование частной виртуальной сети с использованием механизмов туннелирования и криптографической защиты.

Для достижения поставленной цели необходимо решить следующие частные задачи:

1. Реализовать модуль установления соединений между клиентом и сервером с использованием протокола UDP.
2. Разработать модуль взаимодействия с виртуальным сетевым интерфейсом (TUN) для организации туннелирования IP-трафика.
3. Реализовать систему шифрования передаваемых данных с использованием современных криптографических алгоритмов.
4. Разработать механизм аутентификации клиентов с целью обеспечения защищённого доступа к сети.
5. Разработать и реализовать клиентскую и серверную части приложения, обеспечивающие обмен зашифрованным сетевым трафиком.
6. Создать модуль управления сессиями на стороне сервера для отслеживания состояния активных подключений.

7. Реализовать интерфейс командной строки (CLI) для настройки, запуска и управления компонентами системы.
8. Обеспечить совместную работу всех компонентов в едином программном комплексе с возможностью расширения и доработки.

2.2 Описание предметной области

Предметная область курсовой работы охватывает вопросы проектирования и реализации систем виртуальных частных сетей (VPN), предназначенных для обеспечения защищённой передачи данных через открытые или недоверенные сети, такие как Интернет.

В основе функционирования таких систем лежит концепция туннелирования сетевого трафика, при котором данные, предназначенные для одной сети, инкапсулируются в пакеты другого протокола и передаются по защищённому каналу между узлами. Для создания таких каналов используются виртуальные сетевые интерфейсы (например, TUN-интерфейс в операционных системах семейства Linux), позволяющие перехватывать и перенаправлять IP-пакеты, минуя стандартные механизмы маршрутизации.

Дополнительной критически важной составляющей является обеспечение информационной безопасности. Это включает в себя шифрование данных (для защиты конфиденциальности), аутентификацию пользователей (для предотвращения несанкционированного доступа), а также целостность и надёжность передачи информации. Как правило, для этих целей применяются методы асимметричной и симметричной криптографии, в том числе алгоритмы с открытым ключом (например, Ed25519) и шифры блочного или поточного типа (AES, ChaCha20 и др.).

Система поддерживает следующие функции:

1. Генерация криптографических ключей
2. Создание и конфигурация виртуального сетевого интерфейса (TUN)
3. Установление защищённого соединения между клиентом и сервером
4. Шифрование и дешифрование трафика
5. Аутентификация клиентов
6. Управление сессиями подключённых клиентов
7. Маршрутизация IP-пакетов через туннель
8. Интерфейс командной строки (CLI)
9. Гибкая настройка параметров системы

Таким образом, предметная область охватывает все ключевые аспекты организации защищённой виртуальной сети — от генерации криптографических ключей и аутентификации клиентов до туннелирования и шифрования трафика. Разрабатываемое клиент-серверное приложение на языке C++ обеспечивает полный цикл создания и управления приватной сетью, позволяя пользователю безопасно взаимодействовать с удалёнными узлами через недоверенные каналы связи.

2.3 Анализ аналогов

1. WireGuard

WireGuard — это современный VPN-протокол и реализация, отличающаяся высокой производительностью, минимализмом и простотой конфигурации. Протокол построен на современных криптографических примитивах (Curve25519, ChaCha20, Poly1305 и др.) и реализован как модуль ядра Linux, что обеспечивает низкие задержки и высокую пропускную

способность. WireGuard использует концепцию "peer-to-peer", в которой каждый участник сети идентифицируется по публичному ключу.

Преимущества:

- Простая и компактная кодовая база
- Высокая скорость работы
- Интеграция в ядро Linux
- Современная криптография

Недостатки:

- Отсутствие встроенных механизмов авторизации (только по ключам)
- Нет гибкой маршрутизации трафика в стандартной реализации

2. OpenVPN

OpenVPN — один из самых популярных и надёжных VPN-протоколов, широко используемый в корпоративной и частной среде. Он поддерживает большое количество настроек, работает поверх TCP или UDP, а также поддерживает SSL/TLS для шифрования и аутентификации. OpenVPN кроссплатформенен и активно развивается.

Преимущества:

- Поддержка TLS и X.509 сертификатов
- Гибкая конфигурация маршрутов и политик доступа
- Большая экосистема и поддержка

Недостатки:

- Сложность конфигурации
- Более высокая нагрузка по сравнению с WireGuard

- Значительный объём кода

3. SoftEther VPN

SoftEther VPN — это многофункциональная VPN-платформа с открытым исходным кодом, поддерживающая различные протоколы (включая свой собственный, OpenVPN, L2TP/IPsec, SSTP и др.). Обладает графическим интерфейсом и высокой совместимостью с различными операционными системами.

Преимущества:

- Многофункциональность и поддержка разных протоколов
- Высокая совместимость с клиентами Windows, Linux, macOS
- Удобный интерфейс администрирования

Недостатки:

- Сложная архитектура и большая кодовая база
- Затруднённая интеграция в легковесные проекты
- Выше требования к ресурсам

Ключевые отличия создаваемой системы:

- Использование UDP-сокетов для передачи данных
- Аутентификация и генерация ключей на базе Ed25519 с конверсией в X25519
- Конфигурация в формате JSON
- Модульность и открытость архитектуры
- Упрощённый и прозрачный механизм управления сессиями

В ходе анализа популярных VPN-решений — WireGuard, OpenVPN и SoftEther VPN — выявлены их основные преимущества и недостатки с точки зрения производительности, безопасности, удобства конфигурации и архитектурной гибкости. Разрабатываемое в рамках данной работы клиент-серверное приложение ориентировано на сочетание простоты, безопасности и расширяемости. Использование UDP-сокетов и современного криптографического механизма на базе Ed25519 с конверсией в X25519 позволяет обеспечить эффективную и защищённую передачу данных. Применение конфигурационных файлов в формате JSON упрощает настройку и интеграцию, а модульная архитектура облегчает поддержку и развитие системы.

Таким образом, собственное решение позволяет сконцентрироваться на конкретных задачах создания и управления частной виртуальной сетью, сохраняя баланс между функциональностью, производительностью и удобством использования, что делает его актуальным и конкурентоспособным в выбранной предметной области.

2.4. Перечень задач, подлежащих решению в процессе разработки

1. Разработка модуля работы с UDP-сокетами
   1.1. Организация приёма и отправки UDP-пакетов между клиентом и сервером.
   1.2. Обработка ошибок сетевого взаимодействия и повторная отправка пакетов при необходимости.
   1.3. Реализация механизмов таймаутов и контроля доступности соединения.
1. Реализация модуля туннелирования через TUN-интерфейс
   2.1. Создание и инициализация виртуального сетевого интерфейса TUN.
   2.2. Перехват и обработка IP-пакетов, поступающих через TUN-

интерфейс.

2.3. Передача перехваченных пакетов в модуль шифрования и на UDP-сокет.

2.4. Приём и расшифровка пакетов, получение и передача их обратно в сетевой стек через TUN.

2. Разработка модуля шифрования и дешифрования трафика
   3.1. Генерация пары ключей Ed25519 для клиента и сервера.
   3.2. Конвертация ключей Ed25519 в X25519 для обмена общим секретом.
   3.3. Реализация алгоритма симметричного шифрования для защищённой передачи данных.
   3.4. Обеспечение целостности данных и защиты от атак воспроизведения.

3. Реализация модуля аутентификации
   4.1. Проверка подлинности клиента на основе публичного ключа.
   4.2. Управление списком доверенных ключей и политиками доступа.
   4.3. Обработка отказов аутентификации и уведомление пользователя.

4. Разработка клиентской и серверной частей приложения
   5.1. Создание логики подключения, установления и поддержания сессии.
   5.2. Обработка пользовательских команд и взаимодействие с модулями туннелирования и шифрования.
   5.3. Организация корректного завершения сессий и освобождение ресурсов.

5. Реализация модуля управления сессиями на сервере
   6.1. Ведение учёта активных подключений и состояния сессий.
   6.2. Обеспечение безопасного завершения сессий и очистка данных.
   6.3. Логирование событий и статистики работы сервера.

6. Разработка интерфейса командной строки (CLI)
   7.1. Обеспечение запуска и остановки серверной и клиентской частей.
   7.2. Настройка параметров подключения и ключей через конфигурационные файлы JSON.
   7.3. Вывод информации о состоянии подключения, сессиях и ошибках.
7. Тестирование и отладка всех компонентов системы
   8.1. Проведение модульного тестирования каждого из модулей.
   8.2. Выполнение интеграционного тестирования клиент-серверного взаимодействия.
   8.3. Анализ производительности и устойчивости системы при различных нагрузках.

2.5. Обоснование выбора инструментов и платформы для разработки

1. Язык программирования C++:
   - Высокая производительность и эффективность использования ресурсов.
   - Возможность низкоуровневого управления системными вызовами и сетевыми интерфейсами.
   - Широкий набор библиотек для работы с криптографией и сетевыми протоколами.
2. **Система сборки CMake:**
   - Кроссплатформенность и удобство настройки проекта.
   - Автоматизация управления зависимостями и конфигурациями.
   - Простая интеграция с системами непрерывной интеграции и автоматизации.

3. **Контейнеризация с помощью Docker:**
    - Возможность создания изолированных сред для тестирования.
    - Эмуляция работы с несколькими независимыми сетями для проверки взаимодействия клиент-сервер.
    - Упрощение процесса отладки и воспроизведения ошибок в различных сетевых сценариях.

Данные инструменты и платформы были выбраны для обеспечения высокой надёжности и производительности разрабатываемого приложения, а также для удобства разработки, тестирования и дальнейшего масштабирования проекта.

ГЛАВА 3. ПРОЕКТНО-КОНСТРУКТОРСКАЯ ЧАСТЬ.

3.1 Разработка структуры приложения

Модули приложения:

- Модуль работы с UDP-сокетами
- Модуль туннелирования
- Модуль управления устройствами ввода-вывода
- Модуль шифрования
- Модуль аутентификации
- Модуль управления сессиями
- Модуль парсинга конфигурационных файлов
- Клиентская часть
- Серверная часть
- CLI-интерфейс для управления приложением

Коммуникационные протоколы:

- UDP — передача зашифрованных данных между клиентом и сервером

3.2. Разработка алгоритмов обработки информации

Ключевые алгоритмы системы

1. Алгоритм создания и управления UDP-соединением

    1. Инициализация UDP-сокетов на стороне клиента и сервера.

    2. Реализация механизма таймаутов и повторной передачи пакетов для обеспечения надёжности.
    3. Управление состоянием соединения и поддержание активной сессии.
2. Алгоритм туннелирования через TUN-интерфейс
    1. Создание виртуального сетевого интерфейса TUN для перехвата IP-пакетов.
    2. Приём зашифрованных пакетов, их расшифровка и инжекция обратно в сетевой стек через TUN.
3. Алгоритм генерации и обмена ключами на основе Ed25519 и X25519
    1. Генерация пары ключей Ed25519 для каждого участника соединения.
    2. Конвертация ключей Ed25519 в формат X25519 для проведения алгоритма Диффи-Хеллмана.
    3. Совместное вычисление общего секретного ключа для симметричного шифрования сессии.
4. Алгоритм шифрования и дешифрования трафика
    1. Использование общего секретного ключа для симметричного шифрования передаваемых данных ChaCha20.
    2. Дешифрование и проверка полученных пакетов на стороне приёмника.
5. Алгоритм аутентификации клиента
    1. Проверка подлинности клиента по публичному ключу Ed25519.
    2. Сопоставление публичного ключа с доверенным списком на сервере.
    3. Обработка неуспешных попыток аутентификации и ограничение доступа.
6. Алгоритм управления сессиями

1. Ведение учёта активных подключений, их состояния и времени активности.
2. Обработка запросов на установление и завершение сессий.
3. Логирование событий и мониторинг состояния соединений.

3.3. Реализация функционирующего приложения

3.3.1 Реализация модуля UDP-соединения

Одним из ключевых компонентов приложения является модуль, реализующий передачу данных по протоколу UDP. Для этого в проекте был разработан класс `UdpSocket`, инкапсулирующий работу с низкоуровневыми системными вызовами и обеспечивающий интерфейс для отправки и приёма пакетов.

Создание сокета.

В конструкторе `UdpSocket()` создаётся дескриптор сокета с помощью вызова `socket(AF_INET, SOCK_DGRAM, 0)`, указывающего на использование протокола IPv4 и типа сокета UDP. При ошибке выбрасывается исключение. Дополнительно устанавливается опция `SO_REUSEADDR`, позволяющая повторно использовать локальные адреса.

Абстракция над сделующем Posix решением:

```
this->socketFd = socket(AF_INET, SOCK_DGRAM, 0);
setsockopt(socketFd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

Привязка к IP-адресу и порту.

Метод `bind()` позволяет явно задать IP-адрес и порт, к которым должен быть привязан сокет. Используется структура `sockaddr_in`, где задаются семейство адресов, порт и IP в виде сетевого порядка байтов. В случае успешной привязки осуществляется вывод на консоль фактического используемого порта.

Абстракция над сделующем Posix решением:

```
addr.sin_family = AF_INET;
addr.sin_port = htons(bindPort);
inet_pton(AF_INET, bindIp.c_str(), &addr.sin_addr);
::bind(socketFd, (sockaddr*)&addr, sizeof(addr));
```

Неблокирующий режим

Метод `setNonBlocking()` переводит сокет в неблокирующий режим с помощью вызовов `fcntl()`. Это необходимо для асинхронной обработки сетевых событий без блокировки основного потока выполнения.

Абстракция над сделующем Posix решением:

```
int flags = fcntl(this->socketFd, F_GETFL, 0);
fcntl(this->socketFd, F_SETFL, flags | O_NONBLOCK);
```

Отправка и приём данных.

Методы sendTo() и recvFrom() реализуют отправку и прием данных с диагностическими сообщениями в случае ошибки.

Абстракция над сделующем Posix решением:

```
ssize_t sentBytes = sendto(socketFd, buffer, bufferSize, 0, (sockaddr*)&destAddr, sizeof(destAddr));
```

```
ssize_t receivedBytes = recvfrom(socketFd, buffer, bufferSize, 0, (sockaddr*)&srcAddr, &addrLen);
```

3.3.2 Реализация модуля управления виртуальным сетевым интерфейсом

Для реализации туннелирования сетевого трафика в проекте используется виртуальный сетевой интерфейс типа TUN. Управление этим интерфейсом инкапсулировано в классе `TunDevice`, реализующем создание, конфигурацию и удаление интерфейса, а также чтение и запись IP-пакетов.

Создание интерфейса.

Конструктор `TunDevice` осуществляет открытие специального файла `/dev/net/tun`, после чего настраивает параметры интерфейса через системный вызов `ioctl()` с флагами `IFF_TUN | IFF_NO_PI`, указывающими на работу с IP-пакетами без префикса протокола.

Абстракция над следующем Posix решением:

```
this->tunFd = open(TUN_DEVICE, O_RDWR);
ifr.ifr_flags = IFF_TUN | IFF_NO_PI;
strncpy(ifr.ifr_name, tunName.c_str(), IFNAMSIZ - 1);
ioctl(this->tunFd, TUNSETIFF, &ifr);
```

Конфигурация интерфейса.

Конфигурация TUN-интерфейса выполняется через последовательность системных команд:

```
ip addr add <ip>/<netmask> dev <tunName>
sysctl -w net.ipv6.conf.<tunName>.disable_ipv6=1
ip link set dev <tunName> up
sysctl -w net.ipv4.ip_forward=1
```

```
iptables -A FORWARD -i <tunName> -j ACCEPT
iptables -t nat -A POSTROUTING -s <network> -o <eth> -j MASQUERADE
```

Работа с пакетами.

Для работы с пакетами реализованы методы `readPacket()` и `writePacket()`, которые используют стандартные Posix функции `read()` и `write()`.

3.3.3 Реализация модуля управления событиями ввода-вывода

Для эффективной обработки сетевого и TUN-трафика используется механизм неблокирующего ввода-вывода с помощью системного вызова `epoll`, характерного для ОС Linux. Этот модуль инкапсулирует работу с `epoll`, предоставляя абстракцию для регистрации, модификации и удаления файловых дескрипторов из пула наблюдаемых.

Инициализация

При создании объекта `EpollManager` вызывается `epoll_create1(0)`, создавая дескриптор наблюдения за событиями.

Добавление файловых дескрипторов

Метод `addFd(int fd, uint32_t events)` регистрирует дескриптор `fd` с заданными событиями (например, `EPOLLIN`, `EPOLLOUT`, `EPOLLET`).

Изменение маски интересующих событий

Метод `modifyFd(int fd, uint32_t events)` позволяет обновить события, за которыми нужно следить, не удаляя дескриптор из epoll.

Удаление из наблюдения.

Метод `removeFd(int fd)` исключает файловый дескриптор из наблюдаемого списка, освобождая ресурсы.

Ожидание событий.

Метод `wait(epoll_event* events, int maxEvents, int timeoutMs)` — блокирует поток до наступления события или истечения таймаута. Возвращает количество активных дескрипторов.

3.3.4 Реализация модуля управления сессиями

Модуль `SessionManager` отвечает за отслеживание активных пользовательских сессий в VPN-системе. Каждой сессии соответствует уникальный IP-адрес из виртуальной VPN-подсети. Сессия хранит сведения о клиенте и времени его последней активности.

Создание или получение сессии.

Метод `getOrCreateSession(...)` либо возвращает уже существующую сессию, если она найдена по IP-адресу, либо создает новую, если запрос поступает от нового клиента. Создание сопровождается сохранением IP-адреса, UDP-порта и внешнего IP-адреса клиента.

Поиск активной сессии.

Метод `findSessionByVpnIp(...)` возвращает указатель на сессию по VPN IP клиента, если она существует.

Удаление неактивных сессий.

Метод `removeInactiveSessions(...)` периодически очищает хранилище сессий, удаляя устаревшие, которые не проявляли активность в течение заданного тайм-аута. Это предотвращает утечку ресурсов и накопление «мертвых» соединений.

3.3.5 Реализация модуля аутентификации

Модуль `Authenticator` реализует криптографическую аутентификацию клиентов с использованием алгоритма Ed25519 и инфраструктуры OpenSSL. Он используется для проверки подлинности клиента на основе схемы «вызов-ответ» (challenge–response), что исключает передачу приватных ключей и позволяет предотвратить несанкционированный доступ.

Загрузка приватного ключа

Метод `loadPrivateKey(...)` принимает ключ в формате base64, декодирует его в DER и инициализирует объект `EVP_PKEY`. Приватный ключ необходим для подписания данных.

Загрузка публичного ключа

Метод `loadPublicKey(...)` декодирует публичный ключ из base64 и возвращает объект `EVP_PKEY`, который используется для верификации подписи клиента.

Генерация случайного вызова (challenge)

Метод `generateChallenge(...)` формирует случайную последовательность байтов, выступающую в роли задачи для клиента. Размер последовательности задаётся явно.

Подпись вызова

Метод `signChallenge(...)` подписывает challenge с использованием загруженного приватного ключа. Подпись возвращается в виде вектора байтов и может быть отправлена клиенту или серверу для проверки. Проверка подписи вызова

Метод `verifyChallenge(...)` принимает публичный ключ клиента, оригинальный challenge и подпись. Возвращает `true`, если подпись корректна, и `false` в противном случае.

3.3.6 Реализация модуля шифрования

Модуль `Encryption` реализует механизм защищённого обмена сообщениями и формирования общего симметричного ключа по открытым ключам сторон.

Формирование ключа шифрования

Реализуется парсинг ключа ED25519 формата DER, с помощью которого формируется общий секрет, используемый для производства симметричного ключа шифрования HKDF.

Шифрование и дешифрование сообщений

Метод `encrypt(...)` реализует аутентифицированное шифрование с помощью алгоритма ChaCha20-Poly1305.

Метод `decrypt(...)` выполняет обратную операцию и проверяет целостность сообщения по тегу.

3.3.7 Реализация клиентской части

Клиентская часть отвечает за инициализацию VPN-клиента, установление защищённого соединения с сервером и передачу IP-трафика через туннельное устройство TUN.

Основу модуля составляет класс `VpnClient`, реализующий основной цикл обработки событий. В конструкторе класса производится инициализация необходимых компонентов:

- Модуль работы с UDP-сокетами
- Модуль туннелирования
- Модуль управления устройствами ввода-вывода
- Модуль шифрования
- Модуль аутентификации

После инициализации сокета и регистрация дескрипторов интерфейсов в epoll, вызывается метод `eventLoop`, реализующий постоянный цикл обработки входящих и исходящих данных.

Аутентификация осуществляется следующим образом:

1. Клиент отправляет серверу случайный challenge, подписанный своим Ed25519-ключом.
2. Клиент получает ответный challenge сервера.
3. Клиент подписывает ответный challenge и отправляет его серверу.
4. Клиент получает сгенерированный им challenge и проверяет его на подпись сервера.

После успешного прохождения challenge-response механизмов, обе стороны получают возможность вычислить общий симметричный ключ с помощью алгоритма X25519 ECDH и последующего применения функции HKDF-SHA256.

Передача IP-пакетов

Обработка пакетов реализуется в методах `handleRead` и `handleSend`.

Первый вызывается при наличии входящих пакетов из TUN-устройства, шифрует их и пересылает на сервер. Второй вызывается при получении данных по UDP, расшифровывает пакет и передаёт его обратно в TUN-интерфейс.

Передача зашифрованных данных осуществляется по следующей структуре:

```
[1 байт тип сообщения] + [IP-заголовок] + [зашифрованный полезный трафик]
```

3.3.8 Реализация серверной части

Серверная часть отвечает за инициализацию VPN-сервера, установление защищённого соединения с клиентами и передачу IP-трафика через туннельное устройство TUN.

Основу модуля составляет класс `VpnServer`, реализующий основной цикл обработки событий. В конструкторе класса производится инициализация необходимых компонентов:

- Модуль работы с UDP-сокетами
- Модуль туннелирования
- Модуль управления устройствами ввода-вывода
- Модуль шифрования
- Модуль аутентификации
- Модуль управления сессиями

После инициализации сокета и регистрация дескрипторов интерфейсов в epoll, вызывается метод `eventLoop`, реализующий постоянный цикл обработки входящих и исходящих данных.

Аутентификация осуществляется следующим образом:

1. Сервер принимает от клиента challenge.
2. Сервер генерирует ответный случайный challenge, подписанный своим Ed25519-ключом.
3. Сервер подписывает отправленный клиентом challenge и отправляет его клиенту.
4. Сервер принимает ответный challenge и проверяет его на подпись клиента.

После успешного прохождения challenge-response механизмов, обе стороны получают возможность вычислить общий симметричный ключ с помощью алгоритма X25519 ECDH и последующего применения функции HKDF-SHA256.

Передача IP-пакетов

Обработка пакетов реализуется в методах `handleUdpRead` и `handleTunRead`. Первый вызывается при получении данных по UDP, расшифровывает пакет и передаёт его обратно в TUN-интерфейс. Второй вызывается при наличии входящих пакетов из TUN-устройства, шифрует их и пересылает на сервер.

Передача зашифрованных данных осуществляется по следующей структуре:

```
[1 байт тип сообщения] + [IP-заголовок] + [зашифрованный полезный трафик]
```

3.3.9 Реализация модуля парсинга конфигурационных файлов

Модуль `ConfigurationParser` отвечает за чтение и валидацию JSON-конфигурации, определяющей параметры работы клиента или сервера VPN-системы. Он обеспечивает разделение логики по типу конфигурации, проверку корректности полей и предоставление данных в удобной форме для использования другими модулями системы.

Назначение модуля

Задача модуля – извлечение параметров приложения из пользовательского конфигурационного файла формата JSON, предоставляя следующую информацию:

- Тип конфигурации: server или client
- Сетевые параметры: IP-адрес, порт, ключи аутентификации
- Список доверенных клиентов в случае сервера
- Информация о целевом сервере в случае клиента

Основу модуля представляет класс `ConfigurationParser`, реализующего следующие методы:

- Извлечение типа конфигурации

```
std::string parseType(const std::string& configPath)
```

- Извлечение параметров серверного типа конфигурации

```
void parseServerConfiguration(...)
```

- Извлечение параметров клиентского типа конфигурации

```
void parseClientConfiguration(…)
```

3.3.10 Реализация модуля командной строки

Модуль командной строки реализует пользовательский интерфейс командной строки (CLI) для управления VPN-программой. Он обеспечивает обработку аргументов, выполнение ключевых команд (установка пути к конфигурации, генерация ключей, запуск клиента/сервера), а также связывает взаимодействие между конфигурационным парсером и сетевыми компонентами.

Назначение модуля

Модуль служит точкой входа при запуске программы из терминала. Основные задачи:

- Обработка аргументов командной строки;

- Запуск соответствующих процедур: настройка, генерация ключей, запуск VPN;
- Обработка конфигурационного файла;
- Запуск клиента или сервера в зависимости от конфигурации.

Метод `void startCli()`

Реализует диспетчеризацию команд:

- `config` — вызывает `setConfigPath(...)` для записи пути к JSON-файлу конфигурации;
- `genKeys` — генерирует и выводит пару ключей Ed25519;
- `run` — запускает клиент или сервер на основе содержимого конфигурационного файла;
- `help` — выводит справочную информацию.

Если передана неизвестная команда — выводится сообщение об ошибке.

Метод `void setConfigPath(const std::string&)`

Сохраняет путь к конфигурационному файлу во внутреннее хранилище:

- Создаёт директорию `~/.config/myvpn/`, если она не существует;
- Записывает путь в файл `config_path`, откуда он будет загружен при запуске `run`.

Этот подход позволяет пользователю указать путь один раз, без необходимости каждый раз передавать его вручную.

Метод `void generateEncryptionKeys()`

Генерирует пару ключей на основе Ed25519 и выводит их в консоль:

- Приватный ключ;
- Публичный ключ.

Метод `void run()`

Основной метод запуска системы:

- Загружает путь к конфигурационному файлу из `~/.config/myvpn/config_path`;
- С помощью `ConfigurationParser` определяет тип конфигурации;
- В зависимости от типа вызывает `startServer(...)` или `startClient(...)`.

Методы запуска**:**

`void startServer(...)`

Запускает экземпляр `VpnServer`:

- Извлекает IP-адрес и маску;
- Читает порт, приватный ключ и карту пиров;
- Запускает серверный цикл методом `start()`.

`void startClient(...)`

Запускает экземпляр `VpnClient`:

- Извлекает IP-адрес клиента и endpoint сервера;
- Загружает публичный ключ сервера и приватный ключ клиента;
- Запускает цикл обработки пакетов с помощью `eventLoop()`.

Команда	Аргумент	Назначение
`config`	`<path>`	Устанавливает путь к конфигурационному файлу
`genKeys`	–	Генерация парных ключей
`run`	–	Запуск приложение согласно конфигурационному файлу
`help`	–	Вывод справочной информации

Таблица 1. Структура команд

ГЛАВА 4. ПРОЕКТНО-ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ.

4.1 Тестирование и отладка макета рабочей программы

Методология тестирования

В процессе тестирования и верификации функционирования программного обеспечения использовались следующие инструменты:

- Docker — для контейнеризации и моделирования сетевой инфраструктуры;
- tcpdump — для мониторинга и анализа сетевых пакетов;
- ping — для проверки работоспособности сетевого соединения между узлами VPN.

Тестовые сценарии и результаты

Сценарий	Ожидаемый результат	Фактический результат	Статус
Двусторонняя аутентификация	Подтверждение соединения клиента с сервера	Подтверждение соединения клиента с сервера	PASS
Проверка доступности узлов	Получение ответов на ICMP запросы	Получение ответов на ICMP запросы	PASS
Проверка маршрутизации пакетов	Фиксация прохождения ICMP пакетов через тунельный интерфейс	Фиксация прохождения ICMP пакетов через тунельный интерфейс	PASS
Запуск конфигурации без приватного ключа	Ошибка `Missing required keys in 'interface' section (expected: address, listen_port, private_key)`	Ошибка `Missing required keys in 'interface' section (expected: address, listen_port, private_key)`	PASS
Запуск неверной пары ключей аутентификации	Провал аутентификации	Провал аутентификации	PASS

Таблица 2. Тестовые сценарии и результаты

Метрики производительности

Метрика	Значение	Норматив	Соответсвие
Время маршрутизации пакетов	0.38±0.18 мс	≤2 мс	Соответствует
Успешность маршрутизации пакетов	99.99%	99.99%	Соответствует
Способ маршрутизации	Через сервер	Наличие прямой маршрутизации	Отсутсвует

Таблица 3. Метрики производительности

Выявленные проблемы и их решение

1. Проблема: Повышенная задержка при маршрутизации пакетов. Решение: Выделение памяти для буферов данных при запуске, чтобы избежать задержек, связанных с динамическим распределением памяти во время обработки пакетов.
2. Проблема: Отсутсвие прямой маршрутизации.

   Решение: Разработка и внедрение алгоритма распространения маршрутов между клиентами через сервер, с последующей настройкой клиентских маршрутов.

4.2 Разработка руководства пользователя

Назначение

Руководство пользователя предназначено для технических специалистов, таких как системные администраторы и DevOps-инженеры, осуществляющих развертывание, настройку и обслуживание VPN-сети.

Содержание руководства

1. Общие сведения
2. Требования к технической составляющей и архитектуре
3. Работа с CLI
4. Формирование конфигурационного файла
5. Тестирование
6. Безопасность

4.2.1 Общие сведения

Программа представляет собой CLI-интерфейс для создания и управления VPN-соединениями между клиентами и сервером. Основной задачей системы является организация защищённого туннеля между узлами сети с использованием шифрования и аутентификации на основе криптографических ключей.

Решение ориентировано на применение в изолированных или корпоративных сетях, где необходим контроль над маршрутизацией трафика, шифрованием данных и конфигурацией узлов. Поддерживается ручная

настройка конфигурационных файлов, генерация ключей и запуск сетевых компонентов через интерфейс командной строки.

4.2.2 Требования к технической составляющей и архитектуре

Разработанное программное обеспечение предназначено для работы в среде Linux. Его корректная работа на других операционных системах (например, Windows или macOS) не гарантируется и не тестировалась.

Язык программирования и средства сборки:

- Язык программирования: C++ (стандарт C++17)
- Система сборки: Cmake версии 3.14 или выше
- Компилятор: g++ (входит в состав `build-essential`)

Необходимые библиотеки и пакеты:

- buyild-esential — компилятор и инструменты сборки;
- cmake — система конфигурации и сборки проекта;
- libssl-dev — библиотеки OpenSSL для криптографических функций;
- libsodium-dev — криптографическая библиотека libsodium, используемая для работы с Ed25519/X25519;
- pkg-config — утилита для определения параметров сборки зависимостей;
- nlohmann-json3-dev — библиотека для работы с JSON;
- libstdc++6 — стандартная библиотека C++;
- iproute2 — утилиты для настройки маршрутизации и сетевых интерфейсов;

- iptables — инструмент настройки правил фильтрации и маршрутизации пакетов;
- iputils-ping — утилита для проверки доступности сетевых узлов;
- net-tools — набор сетевых утилит (например, ifconfig).

Для отладки и тестирования приложения рекомендуется использовать следующие инструменты:

- tcpdump — для захвата и анализа сетевого трафика;
- Docker — для контейнеризации приложения и управления средой выполнения;
- Docker Compose — для удобного оркестрования и запуска нескольких контейнеров.

4.2.3 Работа с CLI

Генерация ключей

`vpn genKeys` – генерирует и выводит парные ключи ED25519 в формате DER.

```
# Private Key:
# MC4CAQAwBQYDK2VwBCIEIKgmo1BvjP3dzQhXFT8APgwxnvuIGogpNRhFsZ6ctIVn
#
# Public Key:
# MCowBQYDK2VwAyEAmMpkCbq81IkhgqoDSeMa74P32nsHCZFSXAxh10vyW0k=
```

Рекомендуется сохранять сгенерированные ключи в отдельный файл для последующего использования.

Указание пути конфигурационного файла

`vpn config <path>` – сохраняет указанный путь к конфигурационному файлу.

```
# Configuration path “/home/roman/.config/myvpn/server_config” successfully set
```

Запуск приложения

`vpn run` – запускает приложение с заданной конфигурацией, если конфигурационный файл найден.

```
# net.ipv6.conf.vpn_test.disable_ipv6 = 1
# net.ipv4.ip_forward = 1
# TUN device vpn_test created successfully
# Socket bound to port: 5555
```

4.2.4 Фомирование конфигурационного файла

Для корректной работы VPN-приложения необходимо подготовить конфигурационный файл в формате JSON, который задаёт параметры интерфейса и настройки подключения. В зависимости от роли узла — сервера или клиента — структура файла отличается.

Кофигурация клиента:

```
{
  "type": "client",
  "interface": {
    "address": "<виртуальный IP-адрес клиента>/<виртуальная маска>",
    "private_key": "<приватный ключ клиента>"
  },
  "peer": {
    "endpoint": "<IP-адрес сервера>:<порт>",
    "allowed_ips": "0.0.0.0/0",
```

```
    "public_key": "<публичный ключ сервера>"
  }
}
```

Конфигурация сервера:

```
{
  "type": "server",
  "interface": {
    "address": "<виртуальный IP-адрес сервера>/<виртуальная маска сервера>",
    "listen_port": <порт прослушивания>,
    "private_key": "<закрытый ключ сервера>"
  },
  "peers": [
    {
      "allowed_ip": "<виртуальный IP-адрес клиента 1>",
      "public_key": "<публичный ключ клиента 1>"
    },
    {
      "allowed_ip": "<виртуальный IP-адрес клиента 2>",
      "public_key": "<публичный ключ клиента 2>"
    }
  ]
}
```

4.2.5 Тестирование

Тестирование VPN-системы может проводиться как на одном физическом устройстве, так и на нескольких. Оба варианта позволяют проверить корректность работы туннеля, маршрутизации и обмена данными между клиентом и сервером.

Тестирование на одном устройстве

На одном компьютере тестирование удобно выполнять с использованием Docker. Для этого сервер и клиенты запускаются в отдельных контейнерах с настроенными сетевыми интерфейсами.

Dockerfile:

```
FROM ubuntu:22.04

RUN apt update && apt install -y \
    build-essential \
    cmake \
    libssl-dev \
    libsodium-dev \
    pkg-config \
    nlohmann-json3-dev \
    iproute2 iputils-ping net-tools \
    libstdc++6 \
    iptables iproute2 iputils-ping net-tools \
    tcpdump \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY . .
RUN cmake -S . -B build && cmake --build build
WORKDIR /app/build
CMD ["./vpn", "run"]
```

Создание нескольких контейнеров осуществляется последством использования docker-compose.

docker-compose.yaml

```
services:
  vpn_server:
    container_name: vpn_server
    build: .
    cap_add:
      - NET_ADMIN
      - SYS_MODULE
      - NET_RAW
    devices:
      - "/dev/net/tun:/dev/net/tun"
    privileged: true
    volumes:
      - ./server-config.json:/app/config.json
    command: /bin/sh -c "./vpn config /app/config.json && ./vpn run"
    ports:
    - "5555:5555/udp"
    stdin_open: true
    tty: true
    networks:
```

```
      vpn_net:
        ipv4_address: 172.28.0.10

  vpn_client1:
    container_name: vpn_client1
    build: .
    cap_add:
      - NET_ADMIN
      - SYS_MODULE
      - NET_RAW
    devices:
      - "/dev/net/tun:/dev/net/tun"
    privileged: true
    volumes:
      - ./client-config-1.json:/app/config.json
    command: /bin/sh -c "./vpn config /app/config.json && ./vpn run"
    stdin_open: true
    tty: true
    networks:
      vpn_net:
        ipv4_address: 172.28.0.20

networks:
  vpn_net:
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.0.0/16
```

Создаются контейнеры с фиксированными IP-адресами для указания в конфигурационных файлах. Конфигурационные файлы неообходимо создать заранее и указать их относительный путь в параметре `volumes`.

Тестирование на нескольких физических устройствах

В случае использования нескольких физических машин или виртуальных машин, конфигурационные файлы сервера и клиентов размещаются на соответствующих устройствах. Подключение клиента к серверу осуществляется по IP-адресу, указанному в параметре `endpoint`.

Рекомендуемые инструменты:

- ping — для проверки доступности удалённого узла и измерения задержек.
- tcpdump — для анализа сетевого трафика, прохождения ICMP-пакетов и диагностики проблем маршрутизации.

4.2.6 Безопасность

Для обеспечения надежной работы VPN и защиты данных рекомендуется соблюдать следующие меры безопасности:

- **Хранение ключей шифрования**
  Приватные ключи должны храниться в защищённом месте и не передаваться третьим лицам. Рекомендуется сохранять их в файлах с ограниченными правами доступа (например, с помощью chmod 600 на Linux).
- **Резервное копирование конфигурации**
  Файлы конфигурации VPN-сервера и клиентов необходимо регулярно сохранять в безопасном месте для возможности быстрого восстановления работы системы при сбоях.
- **Ограничение доступа к конфигурации**
  Доступ к конфигурационным файлам и ключам должен иметь только уполномоченный персонал.
- **Избегать вывода приватных ключей в общедоступные места**
  Не рекомендуется оставлять приватные ключи в терминале без сохранения в файл, так как это может привести к их утечке.

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы по дисциплине «Сетевые технологии в автоматизированных системах обработки информации и управления» была достигнута поставленная цель — разработка и реализация клиент-серверного приложения, реализующего работу по обеспечению инструментов создания и администрирования частной виртуальной сети.

В ходе выполнения работы были достигнуты следующие цели и решены следующие задачи:

- Разработана архитектура системы с поддержкой клиентского и серверного режимов.
- Реализован модуль командной строки (CLI) для управления приложением.
- Разработан формат конфигурационных файлов на основе JSON.
- Внедрены механизмы аутентификации и шифрования с использованием библиотек OpenSSL и libsodium (алгоритмы Ed25519/X25519).
- Реализованы сетевые компоненты для взаимодействия через UDP и туннельный интерфейс TUN.
- Проведено функциональное тестирование с использованием инструментов `ping` и `tcpdump` в Docker-контейнерах.

В результате выполненной работы получен консольный инструмент для настройки и запуска VPN-соединения на платформе Linux, выполняющий все требования, перечисленные в техническом задании. Выполнение данной курсовой работы позволило не только углубить знания в области сетевых технологий, но и получить практический опыт проектирования и реализации клиент-серверных приложений.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Таненбаум Э. Компьютерные сети. — 5-е изд. — СПб.: Питер, 2013. — 960 с.
2. Kerrisk M. The Linux Programming Interface: A Linux and UNIX System Programming Handbook. — No Starch Press, 2010. — 1552 p.
3. Шлее Дж. CMake на практике. Мастерская программиста. — М.: ДМК Пресс, 2021. — 384 с.
4. Страуструп Б. Язык программирования C++. Специальное издание. — 4-е изд. — М.: Вильямс, 2019. — 1376 с.
5. Краузе А. Linux. Сетевое программирование. — СПб.: Питер, 2018. — 480 с.
6. OpenSSL Project. OpenSSL: The Open Source toolkit for SSL/TLS. [Электронный ресурс]. — URL: https://www.openssl.org/ (дата обращения: 01.06.2025).
7. The CMake Project. CMake Documentation. [Электронный ресурс]. — URL: https://cmake.org/documentation/ (дата обращения: 01.06.2025).
8. libsodium. A modern, easy-to-use crypto library. [Электронный ресурс]. — URL: https://doc.libsodium.org/ (дата обращения: 01.06.2025).
9. nlohmann/json – JSON for Modern C++. [Электронный ресурс]. — URL: https://github.com/nlohmann/json (дата обращения: 01.06.2025).